Portfolio Risk & Scenario Engine

Mathematical Documentation

A detailed exposition of all computations implemented in the codebase

RobinBasso

April 27, 2026

Abstract

This document provides a rigorous mathematical description of every computation performed in the `portfolio-risk-engine` project. The engine is a Python-based framework for simulating multi-asset portfolio dynamics and measuring risk under both normal and stressed market conditions. The four core modules — `portfolio.py`, `returns_model.py`, `monte_carlo.py`, and `risk_metrics.py` — are covered in full detail, followed by the scenario framework defined in `stress_scenarios.py`. All formulae are derived from the source code and correspond exactly to the implemented algorithms.

Contents

1 Notation and Conventions **2**

2 Portfolio Representation (`portfolio.py`) **2**
2.1 Structure . . . 2
2.2 Single-Step Portfolio Return . . . 2
2.3 Portfolio Value Path . . . 3

3 Multivariate Returns Model (`returns_model.py`) **3**
3.1 Model Specification . . . 3
3.2 Covariance Matrix Validation . . . 3
3.3 Simulation Algorithm . . . 4
3.4 Default Calibration (from tests and dashboard) . . . 4

4 Monte Carlo Engine (`monte_carlo.py`) **4**
4.1 Overview . . . 4
4.2 Path Simulation . . . 4
4.3 Final Value Distribution . . . 5

5 Risk Metrics (`risk_metrics.py`) **5**
5.1 Value at Risk (VaR) . . . 5
5.2 Conditional Value at Risk (CVaR / Expected Shortfall) . . . 6
5.3 Maximum Drawdown . . . 6

6 Stress Testing Framework (`scenarios/`) 7
6.1 Design Pattern 7
6.2 Equity Crash Scenario (`EquityCrashScenario`) 7
6.3 Volatility Spike Scenario (`VolatilitySpikeScenario`) 7
6.4 Combined Stress Scenario (`CombinedStressScenario`) 8

7 End-to-End Computational Pipeline 8

8 Portfolio Return Distribution: Analytical Properties 9

9 Module Architecture Summary 9

10 Limitations and Potential Extensions 10

1 Notation and Conventions

Throughout this document we adopt the following notation.

Symbol	Meaning
N	Number of assets in the portfolio
$\boldsymbol{w} \in \mathbb{R}^N$	Portfolio weight vector
V_0	Initial portfolio value (default: 1 000 000)
T	Number of time steps (`n_steps`)
M	Number of simulation paths (`n_sims`)
$\boldsymbol{\mu} \in \mathbb{R}^N$	Vector of per-step expected returns
$\boldsymbol{\Sigma} \in \mathbb{R}^{N \times N}$	Covariance matrix of per-step returns
$\boldsymbol{r}_t \in \mathbb{R}^N$	Vector of asset returns at time step t
R_t	Scalar portfolio return at time step t
V_t	Portfolio value at time step t
α	Confidence level for risk metrics (e.g. 0.95)

All returns in this engine are **simple (arithmetic) returns**, not log-returns. Time steps are abstract and dimensionless; they can represent days, weeks, or any other period depending on how $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ are calibrated.

2 Portfolio Representation (`portfolio.py`)

2.1 Structure

A portfolio is fully characterised by the triple $(N, \boldsymbol{w}, V_0)$ where

$$\boldsymbol{w} = (w_1, w_2, \ldots, w_N)^\top, \qquad \sum_{i=1}^{N} w_i = 1, \qquad w_i \geq 0 \;\; \forall\, i.$$

The two constraints — full investment and no short-selling — are enforced at construction time via `_validate_inputs()`.

2.2 Single-Step Portfolio Return

Given asset returns $\boldsymbol{r}_t = (r_{t,1}, \ldots, r_{t,N})^\top$ at a single time step t, the portfolio return is the weighted inner product:

$$\boxed{R_t = \boldsymbol{w}^\top \boldsymbol{r}_t = \sum_{i=1}^{N} w_i \, r_{t,i}} \tag{1}$$

This is implemented in `portfolio_return` as `np.dot(weights, asset_returns)`.

2.3 Portfolio Value Path

Let $\mathbf{R} \in \mathbb{R}^{T \times N}$ be the returns matrix over T steps, where the t-th row is $\boldsymbol{r}_t^\top$. The scalar portfolio return at each step is:

$$R_t = \mathbf{R}[t, :] \cdot \boldsymbol{w}, \qquad t = 1, \ldots, T. \tag{2}$$

In matrix notation this is simply $\boldsymbol{R}_{\text{port}} = \mathbf{R}\,\boldsymbol{w} \in \mathbb{R}^T$.

The portfolio **value** at time t is computed via the cumulative product of gross returns, starting from the initial value V_0:

$$\boxed{V_t = V_0 \prod_{s=1}^{t} (1 + R_s)} \tag{3}$$

This is the standard compounding formula for arithmetic returns and is implemented as:

```
portfolio_returns = returns_matrix @ self.weights
return self.initial_value * np.cumprod(1 + portfolio_returns)
```

The output array has shape $(T,)$ and contains $[V_1, V_2, \ldots, V_T]$.

Remark 2.1. *The use of* `np.cumprod` *over* $(1 + R_t)$ *is equivalent to the discrete-time wealth process. For small returns* $R_t \ll 1$*, this approximates geometric Brownian motion. There is no log-transform applied; the model operates in the linear return space.*

3 Multivariate Returns Model (`returns_model.py`)

3.1 Model Specification

The engine models asset returns using a **multivariate Gaussian (normal) distribution**:

$$\boxed{\boldsymbol{r}_t \sim \mathcal{N}(\boldsymbol{\mu},\, \boldsymbol{\Sigma})} \tag{4}$$

where the same distribution is applied i.i.d. at every time step. This implies:

- Returns are serially uncorrelated.
- The marginal return of asset i has mean μ_i and variance Σ_{ii}.
- The co-movement between assets i and j is captured by Σ_{ij}.

3.2 Covariance Matrix Validation

The constructor enforces two structural conditions on $\boldsymbol{\Sigma}$:

1. **Squareness:** $\boldsymbol{\Sigma} \in \mathbb{R}^{N \times N}$.
2. **Symmetry:** $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}^\top$ (checked via `np.allclose`).

Positive semi-definiteness is not explicitly checked in the code but is implicitly required for `np.random.multivariate_normal` to produce valid samples.

3.3 Simulation Algorithm

For M paths and T steps, the simulation draws:

$$\boldsymbol{r}_t^{(m)} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}), \qquad m = 1, \ldots, M, \quad t = 1, \ldots, T. \tag{5}$$

The output has shape (M, T, N): M simulations, each containing T rows of N-dimensional return vectors. This is a single call to NumPy's `multivariate_normal` with `size=(n_sims, n_steps)`, which internally uses the Cholesky decomposition $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^\top$ and transforms standard normal draws:

$$\boldsymbol{r}_t = \boldsymbol{\mu} + \boldsymbol{L}\,\boldsymbol{z}_t, \qquad \boldsymbol{z}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}_N). \tag{6}$$

3.4 Default Calibration (from tests and dashboard)

The project uses the following default parameters throughout:

$$\boldsymbol{\mu} = \begin{pmatrix} 0.0005 \\ 0.0002 \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} 0.0001 & 0.00002 \\ 0.00002 & 0.00008 \end{pmatrix}.$$

These correspond to a two-asset portfolio (Equity and Bonds). The implied per-step standard deviations are $\sigma_{\text{Eq}} = 0.01$ (1%) and $\sigma_{\text{Bd}} = 0.00894$ (0.894%), with a correlation of:

$$\rho_{\text{Eq,Bd}} = \frac{\Sigma_{12}}{\sqrt{\Sigma_{11}\,\Sigma_{22}}} = \frac{0.00002}{\sqrt{0.0001 \times 0.00008}} = \frac{0.00002}{0.00002\sqrt{2}} = \frac{1}{\sqrt{2}} \approx 0.707. \tag{7}$$

4 Monte Carlo Engine (`monte_carlo.py`)

4.1 Overview

The `MonteCarloEngine` orchestrates the full simulation pipeline. It combines the `ReturnsModel` and `Portfolio` objects to produce an ensemble of portfolio value paths.

4.2 Path Simulation

For each simulation $m \in \{1, \ldots, M\}$:

1. Draw an $(T \times N)$ matrix of asset returns $\mathbf{R}^{(m)}$ from `returns_model.simulate()`.
2. (Optionally) apply a scenario transformation: $\mathbf{R}^{(m)} \leftarrow \mathcal{S}(\mathbf{R}^{(m)})$ (see Section 6).
3. Compute the portfolio value path using equation (3):

$$V_t^{(m)} = V_0 \prod_{s=1}^{t} (1 + \boldsymbol{w}^\top \boldsymbol{r}_s^{(m)}), \qquad t = 1, \ldots, T.$$

The full output is the matrix $\mathbf{V} \in \mathbb{R}^{M \times T}$, where entry (m, t) is the portfolio value of path m at time step t.

4.3 Final Value Distribution

The method `final_values` extracts the terminal column:

$$\boxed{\widetilde{V}^{(m)} = V_T^{(m)} = V_0 \prod_{t=1}^{T} (1 + R_t^{(m)}), \qquad m = 1, \ldots, M.} \tag{8}$$

This yields a sample $\{\widetilde{V}^{(1)}, \ldots, \widetilde{V}^{(M)}\}$ from the distribution of terminal portfolio wealth, which is then used to compute risk metrics.

Remark 4.1. *Reproducibility is guaranteed via* `np.random.seed(seed)` *before sampling. The default seed used throughout the test suite is 42.*

5 Risk Metrics (`risk_metrics.py`)

All risk metrics are computed from the empirical distribution of terminal portfolio values $\{\widetilde{V}^{(m)}\}_{m=1}^{M}$.

5.1 Value at Risk (VaR)

Definition 5.1 (Value at Risk)**.** *For a confidence level $\alpha \in (0, 1)$, the* ***Value at Risk*** VaR_α *is the minimum loss not exceeded with probability α. Equivalently, it is the loss exceeded with probability $1 - \alpha$.*

The implementation approximates the initial value by the empirical mean of the terminal distribution:

$$\hat{V}_0 = \frac{1}{M} \sum_{m=1}^{M} \widetilde{V}^{(m)}. \tag{9}$$

The VaR is then computed as the gap between this mean and the $(1 - \alpha)$-quantile of the terminal distribution:

$$\boxed{\mathrm{VaR}_\alpha = \hat{V}_0 - q_{1-\alpha}\left(\{\widetilde{V}^{(m)}\}\right)} \tag{10}$$

where $q_p(\cdot)$ denotes the p-th percentile. For $\alpha = 0.95$, this extracts the 5th percentile of terminal values:

$$\mathrm{VaR}_{0.95} = \hat{V}_0 - q_{0.05}\left(\{\widetilde{V}^{(m)}\}\right).$$

Implemented as:

```
initial_value = np.mean(final_values)
percentile = np.percentile(final_values, (1 - alpha) * 100)
return initial_value - percentile
```

Remark 5.1. *Using $\hat{V}_0 = \mathbb{E}[\widetilde{V}]$ rather than the true initial value V_0 is an approximation. For positively drifted processes ($\mu > 0$), $\hat{V}_0 > V_0$, which slightly inflates the reported VaR. A more standard convention would use V_0 directly.*

5.2 Conditional Value at Risk (CVaR / Expected Shortfall)

Definition 5.2 (Conditional Value at Risk)**.** *The **CVaR** at level α (also called* Expected Shortfall*) is the expected loss conditional on the loss exceeding* VaR_α*:*

$$\mathrm{CVaR}_\alpha = \mathbb{E}[\mathit{Loss} \mid \mathit{Loss} \geq \mathrm{VaR}_\alpha]\,.$$

Let $q^* = q_{1-\alpha}(\{\widetilde{V}^{(m)}\})$ be the VaR threshold. The tail set is:

$$\mathcal{T} = \left\{ m : \widetilde{V}^{(m)} \leq q^* \right\}.$$

The CVaR is computed as the mean loss in this tail:

$$\boxed{\mathrm{CVaR}_\alpha = \frac{1}{|\mathcal{T}|} \sum_{m \in \mathcal{T}} \left(\hat{V}_0 - \widetilde{V}^{(m)} \right)} \tag{11}$$

Implemented as:

```
var_threshold = np.percentile(final_values, (1 - alpha) * 100)
tail_losses   = final_values[final_values <= var_threshold]
return np.mean(np.mean(final_values) - tail_losses)
```

CVaR is a **coherent risk measure** (Artzner et al., 1999), satisfying sub-additivity, monotonicity, positive homogeneity, and translation invariance. It is always at least as large as VaR_α by construction: $\mathrm{CVaR}_\alpha \geq \mathrm{VaR}_\alpha$.

5.3 Maximum Drawdown

Definition 5.3 (Maximum Drawdown)**.** *Given a portfolio value path $(V_1, V_2, \ldots, V_T)$, the **maximum drawdown** is:*

$$\mathrm{MDD} = \max_{t \in \{1,\ldots,T\}} \frac{\max_{s \leq t} V_s - V_t}{\max_{s \leq t} V_s}.$$

This measures the largest peak-to-trough decline as a proportion of the running peak. The implementation computes:

$$\boxed{\mathrm{MDD} = \max_t \frac{\mathrm{Peak}_t - V_t}{\mathrm{Peak}_t}} \tag{12}$$

where $\mathrm{Peak}_t = \max_{s \leq t} V_s$ is the running maximum, obtained via `np.maximum.accumulate(path)`:

```
peak      = np.maximum.accumulate(path)
drawdowns = (peak - path) / peak
return np.max(drawdowns)
```

Remark 5.2. *MDD is computed on a* single *simulation path (shape $(T,)$), not on the terminal distribution. In* `test_risk_metrics.py`*, it is applied to the first path of a 1-simulation run.*

6 Stress Testing Framework (`scenarios/`)

6.1 Design Pattern

The scenario framework uses a **Strategy design pattern**. The abstract base class `Scenario` (in `base.py`) defines a single interface method:

$$\mathcal{S} : \mathbb{R}^{T\times N} \to \mathbb{R}^{T\times N}, \qquad \mathbf{R} \mapsto \mathcal{S}(\mathbf{R}).$$

Concrete scenarios override `apply(returns)` to perturb the simulated return matrix before it is passed to the portfolio compounding step.

6.2 Equity Crash Scenario (`EquityCrashScenario`)

An instantaneous additive shock is applied to the equity asset (asset index 0) at the first time step only:

$$\boxed{\widetilde{r}_{1,0}^{(m)} = r_{1,0}^{(m)} + \delta, \qquad \delta = -0.2} \tag{13}$$

with all other steps and assets unchanged:

$$\widetilde{r}_{t,i}^{(m)} = r_{t,i}^{(m)} \quad \text{for } (t,i) \neq (1,0).$$

The parameter $\delta = -0.2$ corresponds to an instantaneous **20% crash** in the equity position at $t = 1$. In code:

```
shocked[0, 0] += self.crash_size    # crash_size = -0.2 by
    default
```

The impact on portfolio value after one step is:

$$V_1^{\text{stressed}} = V_0\big(1 + w_{\text{Eq}}(\mu_{\text{Eq}} + \delta) + w_{\text{Bd}}\, r_{1,\text{Bd}}\big).$$

With the default weights $(w_{\text{Eq}}, w_{\text{Bd}}) = (0.6, 0.4)$ and $\delta = -0.2$, the first-step equity contribution drops by $0.6 \times 0.2 = 0.12$, i.e. a 12% instantaneous portfolio loss.

6.3 Volatility Spike Scenario (`VolatilitySpikeScenario`)

All returns across all time steps and all assets are scaled by a constant factor $\lambda > 1$:

$$\boxed{\widetilde{\mathbf{R}}^{(m)} = \lambda \cdot \mathbf{R}^{(m)}} \tag{14}$$

The default factor is $\lambda = 2$. This multiplicative scaling has the following effect on the moments of the portfolio return distribution:

$$\mathbb{E}[\widetilde{R}_t] = \lambda\, \mathbb{E}[R_t] = \lambda\, \boldsymbol{w}^\top \boldsymbol{\mu}, \tag{15}$$

$$\mathrm{Var}[\widetilde{R}_t] = \lambda^2\, \mathrm{Var}[R_t] = \lambda^2\, \boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w}. \tag{16}$$

Thus volatility (standard deviation) is multiplied by λ, while expected return is also scaled. This is a *symmetric* stress: both upside and downside paths are amplified equally.

6.4 Combined Stress Scenario (`CombinedStressScenario`)

This scenario applies both an equity crash and a volatility spike sequentially:

$$\widetilde{\mathbf{R}}^{(m)} = \lambda \cdot \mathcal{S}_{\text{crash}}(\mathbf{R}^{(m)}), \qquad \lambda = 1.5, \;\; \delta = -0.2. \tag{17}$$

Explicitly:

$$\boxed{\widetilde{r}_{t,i}^{(m)} = \begin{cases} 1.5\,(r_{1,0}^{(m)} - 0.2) & \text{if } t = 1,\, i = 0 \\ 1.5\, r_{t,i}^{(m)} & \text{otherwise} \end{cases}} \tag{18}$$

In code:

```
shocked[0, 0] -= 0.2    # equity crash at t=0
shocked *= 1.5           # scale all returns
```

Remark 6.1. *The ordering matters: the crash shock is applied* before *the scaling, so the crash magnitude itself is also amplified. The effective first-step equity shock becomes* $1.5 \times (-0.2) = -0.30$*, i.e. a 30% crash on the equity position at* $t = 1$*.*

7 End-to-End Computational Pipeline

The full computation performed for a single simulation run can be summarised as follows. Let M be the number of paths, T the number of steps, N the number of assets.

Step 1 Asset return sampling.

$$\mathbf{R}^{(m)} \sim \mathcal{N}_{T\times N}(\mathbf{1}_T \boldsymbol{\mu}^\top, \boldsymbol{\Sigma}), \quad m = 1, \ldots, M.$$

Step 2 Scenario perturbation (optional).

$$\mathbf{R}^{(m)} \leftarrow \mathcal{S}(\mathbf{R}^{(m)}).$$

Step 3 Scalar portfolio returns.

$$\boldsymbol{R}_{\text{port}}^{(m)} = \mathbf{R}^{(m)} \boldsymbol{w} \in \mathbb{R}^T.$$

Step 4 Portfolio value paths.

$$V_t^{(m)} = V_0 \prod_{s=1}^{t} (1 + R_{\text{port},s}^{(m)}), \quad t = 1, \ldots, T.$$

Step 5 Terminal value extraction.

$$\widetilde{V}^{(m)} = V_T^{(m)}, \quad m = 1, \ldots, M.$$

Step 6 Risk metrics computation.

$$\begin{aligned} \hat{V}_0 &= \frac{1}{M} \sum_{m=1}^{M} \widetilde{V}^{(m)}, \\ \text{VaR}_\alpha &= \hat{V}_0 - q_{1-\alpha}(\{\widetilde{V}^{(m)}\}), \\ \text{CVaR}_\alpha &= \hat{V}_0 - \frac{1}{|\mathcal{T}|} \sum_{m \in \mathcal{T}} \widetilde{V}^{(m)}, \quad \mathcal{T} = \{m : \widetilde{V}^{(m)} \le q_{1-\alpha}\}. \end{aligned}$$

8 Portfolio Return Distribution: Analytical Properties

Under the Gaussian returns model, the single-step portfolio return $R_t = \boldsymbol{w}^\top \boldsymbol{r}_t$ is itself Gaussian:

$$R_t \sim \mathcal{N}\big(\mu_p,\, \sigma_p^2\big)\,, \tag{19}$$

where:

$$\mu_p = \boldsymbol{w}^\top \boldsymbol{\mu}, \qquad \sigma_p^2 = \boldsymbol{w}^\top \boldsymbol{\Sigma} \boldsymbol{w}. \tag{20}$$

For the default parameterisation with $\boldsymbol{w} = (0.6, 0.4)^\top$:

$$\begin{aligned}
\mu_p &= 0.6 \times 0.0005 + 0.4 \times 0.0002 = 0.000380, && (21)\\
\sigma_p^2 &= (0.6)^2 \times 0.0001 + 2 \times 0.6 \times 0.4 \times 0.00002 + (0.4)^2 \times 0.00008 \\
&= 0.000036 + 0.0000096 + 0.0000128 = 0.0000584, && (22)\\
\sigma_p &\approx 0.00764. && (23)
\end{aligned}$$

The terminal portfolio value $V_T = V_0 \prod_{t=1}^{T}(1 + R_t)$ is the product of T i.i.d. random variables. For large T and small returns, this approximates a **log-normal distribution** by the central limit theorem applied to log-returns:

$$\log V_T \approx \log V_0 + T\big(\mu_p - \tfrac{1}{2}\sigma_p^2\big) + \sigma_p \sqrt{T}\, Z, \quad Z \sim \mathcal{N}(0, 1).$$

The empirical histogram plotted in the Streamlit dashboard reflects this approximately log-normal shape.

9 Module Architecture Summary

Table 1: Summary of modules and their mathematical roles

Module	**Class / Method**	**Mathematical Operation**
`portfolio.py`	`portfolio_return`	$R_t = \boldsymbol{w}^\top \boldsymbol{r}_t$
	`portfolio_path`	$V_t = V_0 \prod_{s \le t}(1 + R_s)$
`returns_model.py`	`simulate`	$\boldsymbol{r}_t \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$
`monte_carlo.py`	`simulate`	Full path ensemble $\mathbf{V} \in \mathbb{R}^{M \times T}$
	`final_values`	Terminal slice $\widetilde{V}^{(m)} = V_T^{(m)}$
`risk_metrics.py`	`var`	$\hat{V}_0 - q_{1-\alpha}$
	`cvar`	Mean loss in tail $\mathcal{T}$
	`max_drawdown`	$\max_t(\text{Peak}_t - V_t)/\text{Peak}_t$
`stress_scenarios.py`	`EquityCrashScenario`	$r_{1,0} \mathrel{+}= \delta$
	`VolatilitySpikeScenario`	$\mathbf{R} \leftarrow \lambda \mathbf{R}$
	`CombinedStressScenario`	Crash then scale

10 Limitations and Potential Extensions

The current implementation makes several simplifying assumptions worth noting:

1. **Gaussian returns.** The model does not capture fat tails, skewness, or volatility clustering (e.g. GARCH effects). Real asset returns exhibit kurtosis significantly greater than 3.
2. **Fixed weights.** The portfolio is assumed to be statically allocated with no rebalancing. Dynamic strategies (momentum, mean-reversion) are not modelled.
3. **I.i.d. steps.** Returns are independent across time steps. Serial correlation and regime-switching are absent.
4. **VaR approximation.** Using $\hat{V}_0 = \mathbb{E}[\widetilde{V}]$ rather than the true V_0 introduces a small upward bias in all risk metrics.
5. **Scenario scope.** Stress scenarios modify only the simulated return matrix rather than recalibrating $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, which is a reduced-form rather than structural approach.
6. **No discounting.** All values are nominal; there is no risk-free rate or discounting applied to future cash flows.

Natural extensions would include: Student-t copulas for fat-tailed dependence, GARCH(1,1) volatility dynamics, Cholesky-based correlation shocks for multi-factor stress scenarios, and CVaR-based portfolio optimisation.

References

- Artzner, P., Delbaen, F., Eber, J.-M., & Heath, D. (1999). *Coherent measures of risk.* Mathematical Finance, 9(3), 203–228.
- McNeil, A.J., Frey, R., & Embrechts, P. (2015). *Quantitative Risk Management: Concepts, Techniques and Tools* (Revised ed.). Princeton University Press.
- Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering.* Springer.
- Rannacher, R. (1984). Finite element solution of diffusion problems with irregular data. *Numerische Mathematik*, 43, 309–327.